

# 정답지

인출 훈련판용 — 모든 칸을 쓴 뒤에만 확인할 것.

## 01 · TCP vs UDP

(1) TCP는 연결을 먼저 맺고 확인응답과 재전송으로 신뢰성과 순서를 보장하는 대신 오버헤드가 크고, UDP는 연결 없이 그냥 보내서 빠른 대신 아무것도 보장하지 않는다.

한 문장 모델 · ●●

(2) '오류 검출 기능이 전혀 없다' — 체크섬은 있으므로 오답

UDP · 시험 함정 · ●●●

(3) 20바이트 (옵션 제외)

TCP · 헤더 크기 · ●●●

(4) 확인응답(ACK) + 재전송으로 보장

TCP · 신뢰성 · ●●

(5) 보장하지 않음

UDP · 순서 보장 · ●●

(6) 8바이트

UDP · 헤더 크기 · ●●●

(7) 시퀀스 번호로 보장

TCP · 순서 보장 · ●●

(8) 연결형 — 3-way handshake

TCP · 연결 방식 · ●●

(9) 데이터를 보내기 전에 연결을 맺는가. 맺으면 상태를 유지할 수 있으므로 확인응답 · 재전송 · 순서 · 흐름 제어가 전부 가능해지고, 맺지 않으면 전부 불가능하다.

결정적 변별 · ●●●

(10) 기능이 많은 쪽이 헤더가 크다 — TCP가 20.

TCP 헤더 20바이트 / UDP 헤더 8바이트 · ●●●

(11) UDP는 검출만 하고 버린다. 복구(재전송)는 TCP만 한다.

오류 검출 / 오류 복구 · ●●●

(12) UDP 헤더에도 체크섬 필드가 있어 오류 검출은 수행한다

UDP는 오류 검출 기능이 없다. · ●●●

(13) 손실률이 매우 높은 환경에서는 UDP 재전송 구현이 더 느릴 수 있다

TCP는 항상 UDP보다 느리다. · ●●●

(14) 연결은 3-way, 종료는 4-way handshake

TCP 연결 종료는 3-way handshake로 이루어진다. · ●●●

## 02 · CPU 스케줄링 알고리즘

(15) 스케줄링 알고리즘은 CPU를 뺏을 수 있는가(선점)와 특정 프로세스가 영원히 밀릴 수 있는가(기아)라는 두 축으로 구분된다.

한 문장 모델 · ●●

(16) 쿼텀이 너무 작으면 문맥 교환 비용 폭증

RR · 한계 · ●●●

(17) 발생

SRT · 기아 발생 · ●●●

(18) 해결책은 에이징(aging)

우선순위 · 한계 · ●●●

(19) 문맥 교환 오버헤드 증가

SRT · 한계 · ●●●

(20) 실행시간을 미리 알 수 없어 실제 적용이 어려움

SJF · 한계 · ●●●

(21) 선점 · 비선점 모두 가능

우선순위 · 선점 여부 · ●●

(22) 공평하다는 이유로 평균 대기시간도 좋다고 착각

FCFS · 시험 함정 · ●●●

(23) 비선점

FCFS · 선점 여부 · ●●

(24) 발생 — 낮은 우선순위가 무한 대기

우선순위 · 기아 발생 · ●●●

(25) 기아 해결책을 '선점 전환'으로 적은 선지

우선순위 · 시험 함정 · ●●●

(26) 비선점

SJF · 선점 여부 · ●●

(27) 호위 효과 — 긴 작업이 앞서면 전체 지연

FCFS · 한계 · ●●●

(28) 최적이라는 서술만 보고 실현 가능하다고 판단

SJF · 시험 함정 · ●●●

(29) 쿼텀이 매우 커지면 FCFS와 같아진다는 점

RR · 시험 함정 · ●●●

(30) 실행 중인 프로세스에서 CPU를 뺏을 수 있는가. 뺏을 수 있으면 선점형이고 응답성이 좋아지는 대신 문맥 교환 비용이 늘어난다.

결정적 변별 · ●●●

(31) 이름에 Remaining이 있으면 선점형이다.

SJF / SRT · ●●●

(32) 타임 쿼텀이 언급되면 RR이다. 쿼텀이 무한대면 FCFS와 동일.

FCFS / RR · ●●●

(33) 실행시간을 알 수 없으므로 실제 시스템에서는 적용 불가

SJF는 항상 최적의 스케줄링이다. · ●●●

(34) 모든 프로세스가 순환하므로 기아는 발생하지 않는다

라운드 로빈에서는 기아가 발생한다. · ●●●